```
cm@cm-Vostro-3559:~/osfydec2k19$ vim noncompliantatomic.c
cm@cm-Vostro-3559:~/osfydec2k19$ vim compliantatomic.c
cm@cm-Vostro-3559:~/osfydec2k19$ gcc noncompliantatomic.c
cm@cm-Vostro-3559:~/osfydec2k19$ ./a.out
sigerrmsg in main before raise SIGINT Raised
sigerrmsg in main after raise SIGINT Received
cm@cm-Vostro-3559:~/osfydec2k19$ gcc compliantatomic.c
cm@cm-Vostro-3559:~/osfydec2k19$ ./a.out
sigerrmsg in main before raise SIGINT Raised
sigerrmsg in main after raise SIGINT Received
cm@cm-Vostro-3559:~/osfydec2k19$
```

Figure 3: Output of compliant and non-compliant atomic code

```
cm@cm-Vostro-3559:~/osfydec2k19$ vim callbackstruct.c
cm@cm-Vostro-3559:~/osfydec2k19$ gcc callbackstruct.c
cm@cm-Vostro-3559:~/osfydec2k19$ ./a.out
gstatus value in write 20
gstatus value in read 20
cm@cm-Vostro-3559:~/osfydec2k19$
```

Figure 4: Output of read/writes with a structure function pointer

CPU instruction cycle. This consists of fetching the instruction code from memory [PC], and decoding the following instruction:

```
Mull    x, y, product
```

The above example reveals that it's a multiply instruction, and that the operands are memory locations x, y, and product. So you fetch x and y from memory, multiply x and y, store the result in a CPU register, and save the result from the CPU to the memory location product.

MIPS (Microprocessor without Interlocked Pipelined Stage) is load-store architecture where all instructions except *load* (copy from memory to CPU register) and *store* (copy from CPU register to memory) get their operands from CPU registers and store their results in a CPU register. Hence, the instruction cycle for all instructions except *load* and *store* is somewhat simpler. When all operands are in CPU registers, which can be accessed within a single clock cycle, fetching operands and storing the results can occur within the same clock cycle as execution (add, subtract, etc). For example, let's suppose R0, R1, R2 ... R15 are CPU registers. Then the operation:

```
R0 ← R4 + R7       # one clock cycle
```

…is a simple, atomic operation inside the CPU, and therefore is not regarded as multiple steps in the instruction cycle. Basically, if an instruction cycle completes in a single CPU clock cycle, then it is an atomic operation.

Signal handler

A signal is a software generated interrupt that is sent to a process by the OS when users press *Ctrl-c* or when another process tells something else to this process. If a process wishes to handle certain signals, then in the code, the process must register a signal handling function to the kernel. The signal handler function has 'void return' type and accepts a signal number corresponding to the signal that needs to be handled. To get the signal handler function registered to the kernel, the signal handler function pointer is passed as the second argument to the 'signal' function.

GCC has the system call *raise ()* for sending a signal to the program that contains this *raise ()* call. Note that this system call can only send a signal to the program that contains it. *raise ()* cannot send a signal to other processes. If this function is executed successfully, then the signal specified in *sig* is generated. If the program has a signal handler, this signal will be processed by that signal handler; otherwise, this signal will be handled in the default way.

Accessing or modifying shared objects in signal handlers can result in race conditions that can leave data in an inconsistent state. The two exceptions to this rule are the ability to read from and write to lock-free atomic objects and variables of type *volatile sig_atomic_t*. Accessing any other type of object from a signal handler is undefined behaviour.

An example of a signal handler with global memory *[Char *sigerrmsg]* without an atomic object is given below:

```
//noncompliantatomi.c
#include<stdio.h>
#include<signal.h>
#include<stdlib.h>
#include<string.h>

#define MSGSIZE 24
char *sigerrmsg;
void handler (int signum)
{
    strcpy(sigerrmsg,"SIGINT
Received");
}

int main()
{
    signal(SIGINT,handler);
    sigerrmsg=(char *)malloc(MSGSIZE);
    if(!sigerrmsg) {
        printf("Memory not
allocated\n");
        exit(0);
```